SPRAWOZDANIE Z ZADANIA 2

TEMAT:

Tester sprawności psychomotorycznej kandydatów na kierowców

Imię i nazwisko: Cyprian Magnucki 197697, Małgorzata Rogala 198123

Grupa: A2A

Przedmiot: Organizacja Systemów Komputerowych - Laboratorium

Data: 24.03.2026 r.

1. Treść zadania

Tematem zadania drugiego jest napisanie aplikacji uzależnionej od czasu. Jednym z zadań aplikacji ma być możliwie precyzyjny pomiar lub odmierzanie czasu. Wykorzystując dowolny język programowania dla komputerów w standardzie PC napisać aplikację spełniającą funkcję testera sprawności psychomotorycznej np. kandydatów na kierowców. Na aplikację powinna się składać seria różnych testów badających prosty i złożony czas reakcji na bodźce optyczne i akustyczne. Każdy test właściwy powinna poprzedzać informacja o przebiegu testu oraz faza szkoleniowa, w trakcie której osoba badana wykona te same czynności co w trakcie testu, ale bez oceny. Po wykonaniu serii testów osoba poddana badaniom powinien zostać poinformowana o osiągniętych wynikach w formie syntetycznej i analitycznej z wykorzystaniem wartości liczbowych i reprezentacji graficznej.

2. Założenia projektowe - dokładne sformułowanie podjętego zadania

W ramach realizacji projektu przyjęto następujące założenia:

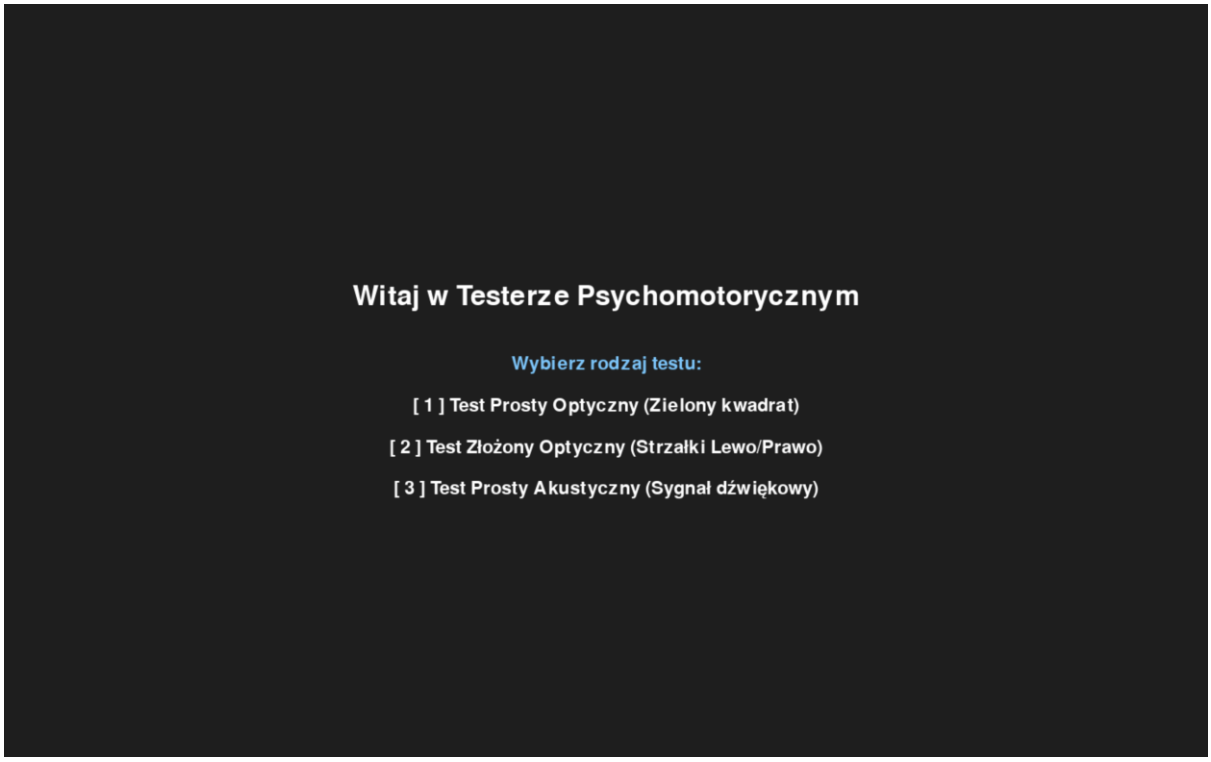
- Aplikacja została wykonana w języku **Python**,
- Do stworzenia interfejsu graficznego, obsługi okien, rysowania kształtów i generowania dźwięku wykorzystano bibliotekę **Pygame**,
- Program działa w trybie okienkowym z możliwością swobodnej zmiany rozmiaru okna oraz przejścia w natywny tryb pełnoekranowy (fullscreen) w celu minimalizacji rozpraszaczy,
- Sterowanie aplikacją odbywa się za pomocą klawiatury (klawisze: `Spacja`, `1`, `2`, `3`, `Strzałki Kierunkowe`, `ESC`),

- Aplikacja zawiera serię trzech testów:
    - **Test prosty optyczny:** reakcja na zielony kwadrat (spacja).
    - **Test złożony optyczny:** reakcja kierunkowa na pojawiające się strzałki (strzałka w lewo/prawo).
    - **Test prosty akustyczny:** reakcja na sygnał dźwiękowy bez bodźca wzrokowego (spacja).
- Każdy z testów posiada obligatoryjną, nieocenianą fazę szkoleniową (3 próby) oraz fazę testu właściwego (5 prób),
- Interwał czasowy oczekiwania na bodziec w każdej próbie jest generowany losowo z przedziału od 2 do 5 sekund,
- Pomiar czasu realizowany jest przy użyciu precyzyjnego licznika sprzętowego wysokiej rozdzielczości,
- Po zakończeniu testu aplikacja generuje syntetyczno-analityczny raport z wynikami (średnia, wartości skrajne, odchylenie standardowe, skuteczność) oraz przedstawia wyniki na wykresie liniowym.

3. Opis działania aplikacji

3.1 Główne menu programu

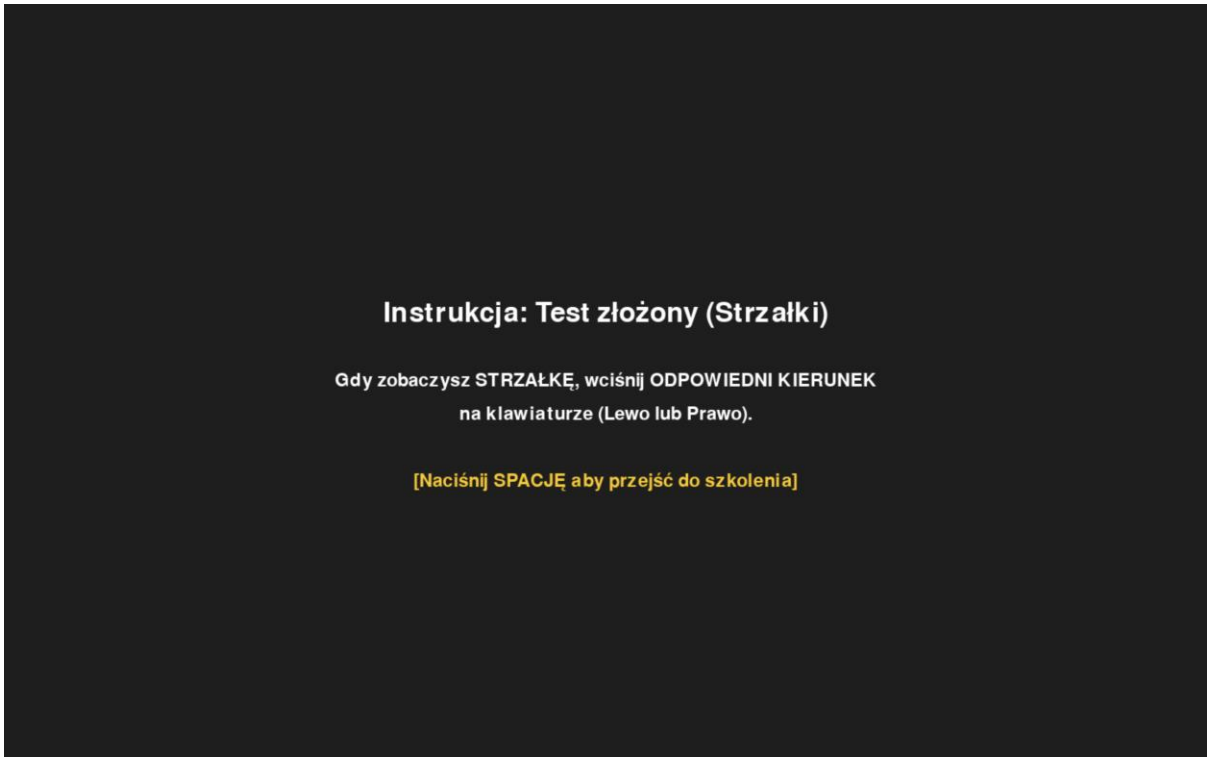
Po uruchomieniu programu użytkownik wita centralnie wyśrodkowane okno z Menu Głównym. Aplikacja oczekuje na wybór jednego z trzech dostępnych testów za pomocą klawiszy numerycznych (1, 2 lub 3). W tym miejscu użytkownik może również rozszerzyć okno na pełny ekran korzystając z systemowych przycisków okna.



Rysunek 1: Menu główne aplikacji z wyborem testów

3.2 Ekran instrukcji i faza szkoleniowa

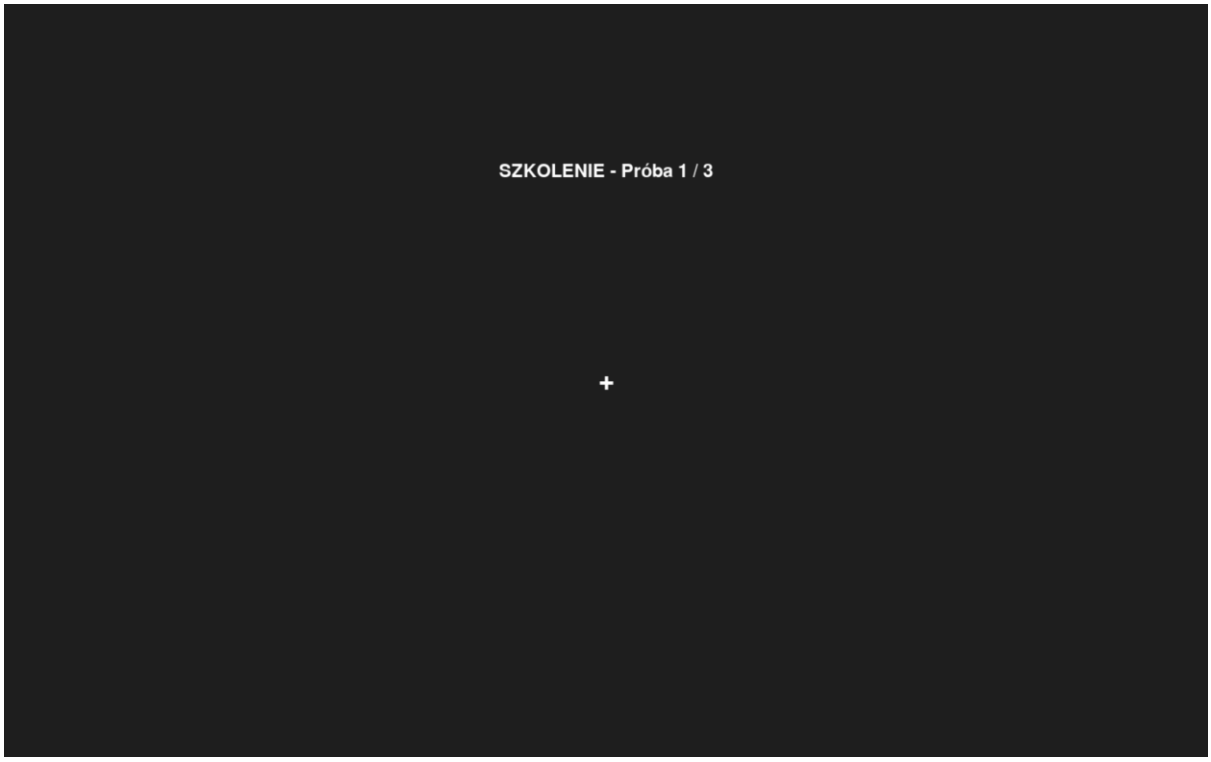
Po wyborze konkretnego testu, na ekranie wyświetlana jest spersonalizowana instrukcja przypisana do wybranego trybu (np. informująca o konieczności użycia strzałek zamiast spacji dla testu złożonego). Naciśnięcie spacji rozpoczyna fazę szkoleniową. Składa się ona z 3 prób. Na ekranie widnieje krzyżyk skupienia, a po losowym czasie pojawia się bodziec (lub odtwarzany jest dźwięk). Po każdej próbie szkoleniowej na ekranie na krótki czas (1.5s) wyświetlany jest uzyskany wynik lub informacja o błędzie (falstart), z wyraźnym zaznaczeniem, że wynik ten nie jest zapisywany.



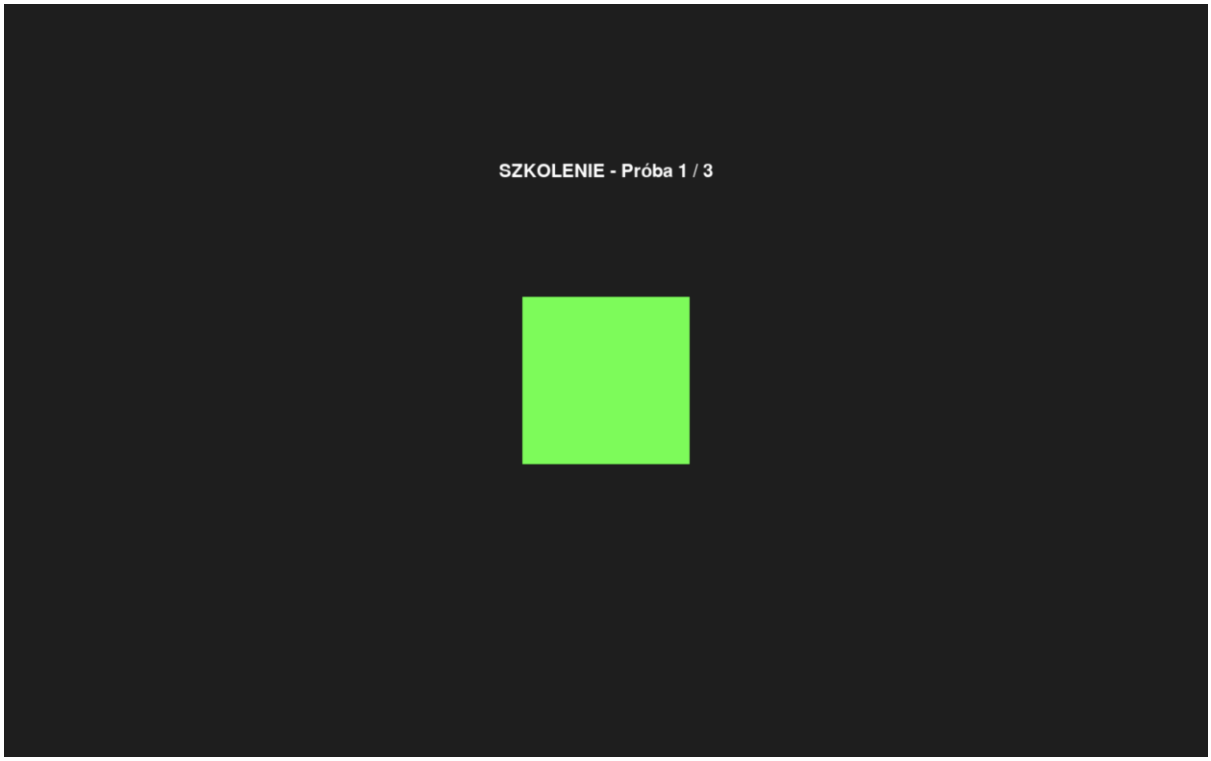
Rysunek 2: Ekran instrukcji dla testu złożonego

3.3 Faza testu właściwego

Faza ta nie różni się wizualnie od fazy szkoleniowej, jednak składa się z 5 prób, a zmierzone w niej czasy reakcji są zapisywane do wewnętrznej bazy danych programu (tablicy). Program zlicza również liczbę popełnionych falstartów oraz (w przypadku testu złożonego) liczbę błędnych decyzji kierunkowych.

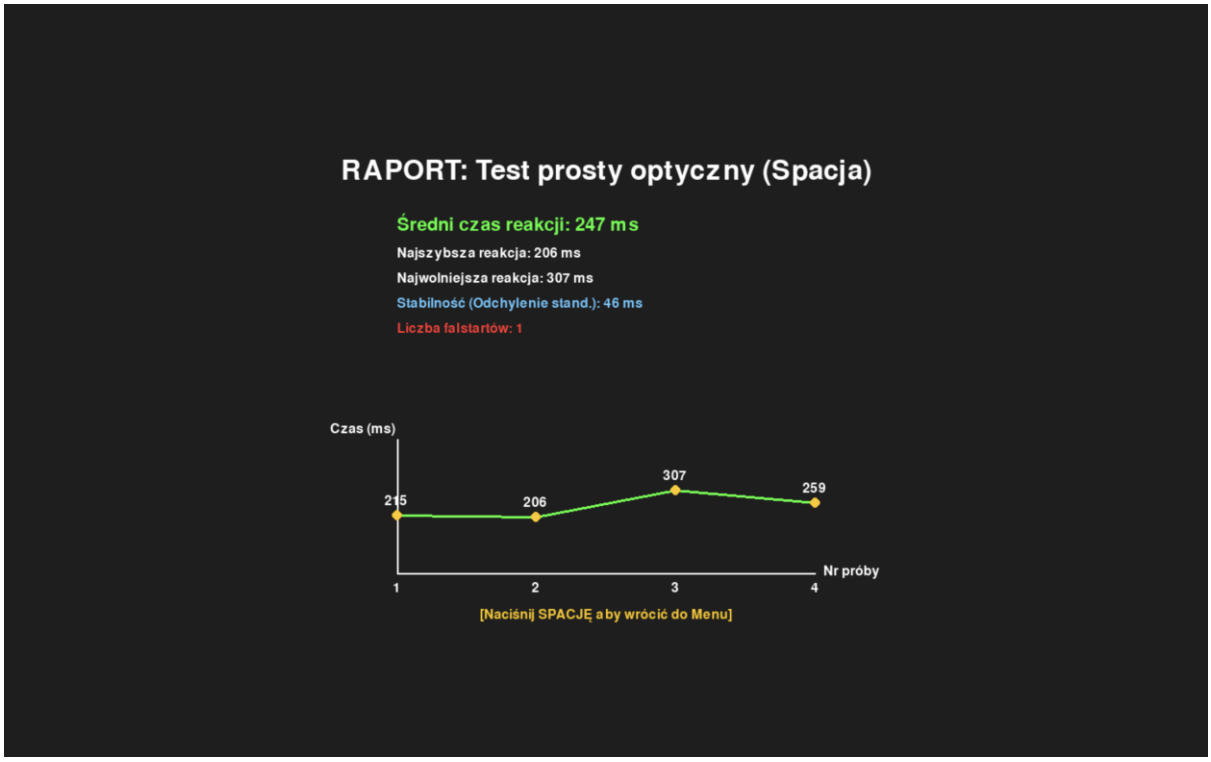


Rysunek 3: Faza oczekiwania na bodziec oraz sam bodziec optyczny



3.4 Ekran raportu i analityki

Po zrealizowaniu 5 prób testu właściwego, automatycznie generowany jest ekran wyników. Przedstawia on formę syntetyczną (kolorystyczna ocena, czy średni czas znajduje się w normie) oraz analityczną. W części tekstowej prezentowane są: średni czas, najszybsza i najwolniejsza reakcja, stabilność układu nerwowego (odchylenie standardowe) oraz procentowa poprawność decyzji. W dolnej części ekranu rysowany jest wykres liniowy obrazujący "krzywą zmęczenia" badanego w kolejnych próbach.



Rysunek 4: Ekran raportu z wygenerowanym wykresem liniowym

4. Opis zastosowanych rozwiązań programowych

4.1 Wybór języka i biblioteki

Do realizacji projektu wykorzystano język Python ze względu na jego elastyczność i bogaty ekosystem modułów matematycznych. Do budowy interfejsu graficznego i obsługi multimediów wybrano bibliotekę **Pygame**. W odróżnieniu od bibliotek okienkowych (takich jak Tkinter), Pygame pozwala na bezpośredni dostęp do pętli zdarzeń i buforów ekranu, co jest kluczowe dla minimalizacji opóźnień (input lag) i utrzymania rygorystycznego rygoru czasowego aplikacji.

4.2 Struktura programu

Struktura programu opiera się na obiektowym wzorcu **Maszyny Stanów (State Machine)**. Klasa wyliczeniowa `AppState`przechowuje możliwe stany globalne (`MENU`, `INSTRUCTION`, `TRAINING`, `TEST`, `RESULTS`). Klasa główna aplikacji `PsychoTesterApp` posiada dodatkowo tzw. "podstany" (`WAITING`, `STIMULUS`, `FEEDBACK`), które zarządzają mikrocyklem pojedynczej próby pomiarowej.

4.3 Przykładowe funkcje programu

Tabela 1: Wybrane metody klasy PsychoTesterApp i ich przeznaczenie

Funkcja	Przeznaczenie
__init__()	Inicjalizacja silnika Pygame, ustawienie parametrów okna, inicjalizacja zmiennych stanu i bufora audio.
generate_beep()	Generowanie fali sinusoidalnej dla bodźca akustycznego w pamięci RAM.
start_new_trial()	Inicjalizacja pojedynczej próby (losowanie czasu opóźnienia, reset stoperów).
handle_events()	Przechwytywanie zdarzeń systemowych (klawiatura, zmiana rozmiaru okna). Logika pomiaru czasu po wciśnięciu klawisza.
update()	Aktualizacja timerów i zarządzanie płynnym przejściem między podstanami (np. oczekiwanie -> bodziec).
draw()	Renderowanie odpowiednich elementów graficznych w zależności od aktualnego stanu i podstanu.
draw_graph()	Obliczanie skali, skalowanie wektorowe i rysowanie wykresu liniowego na podstawie tablicy wyników.
advance_state()	Przenoszenie aplikacji do kolejnego głównego stanu po zakończeniu cyklu.

4.4 Opis kluczowych funkcji pomiarowych

Funkcja `handle_events()` jest najważniejszym punktem programu z perspektywy pomiarowej. Pętla przechwytuje zdarzenie `pygame.KEYDOWN`. Jeśli nastąpi ono w podstanie `WAITING` (przed bodźcem), rejestrowany jest **falstart**. Jeśli w podstanie `STIMULUS`, funkcja natychmiast zatrzymuje stoper.

```
elif self.sub_state == "STIMULUS":
    # Pobranie precyzyjnego czasu w momencie wciśnięcia
    end_time = time.perf_counter()
    valid_key_pressed = False
    is_correct = False

    # Obsługa logiki dla różnych testów
    if self.test_mode in [1, 3]: # Test prosty
        if event.key == pygame.K_SPACE:
            valid_key_pressed = True
            is_correct = True
    elif self.test_mode == 2: # Test złożony (strzałki)
        if event.key in [pygame.K_LEFT, pygame.K_RIGHT]:
            valid_key_pressed = True
```

```
            if event.key == pygame.K_LEFT and self.current_arrow_dir
== "LEFT":
                is_correct = True
            elif event.key == pygame.K_RIGHT and
self.current_arrow_dir == "RIGHT":
                is_correct = True

    if valid_key_pressed:
        # Obliczenie czasu w ms i zapisanie do tablicy
        self.last_reaction_time = (end_time -
self.stimulus_start_time) * 1000
        self.last_was_correct = is_correct
        if self.state == AppState.TEST:
            self.results.append(self.last_reaction_time)
```

Aby zapobiec nieuczciwym pomiarom lub tzw. *hardware latency*, w funkcji `update()` (kontrolującej logikę), stoper dla testów wizualnych uruchamiany jest **dopiero w momencie**, w którym flaga `visual_frame_drawn` potwierdzi, że monitor faktycznie wyrenderował klatkę z kwadratem lub strzałką.

4.5 Optymalizacja audio i wyzwania systemowe

Rozwiązanie problemu opóźnienia przy odtwarzaniu bodźców akustycznych stanowiło wyzwanie programowe. Standardowe odtwarzanie plików z dysku oraz systemowe miksery wprowadzają narzut czasowy sięgający nawet 200 milisekund (szczególnie w systemie macOS). Rozwiązano to wymuszając inicjalizację miksera w trybie natywnym (Stereo, 44.1kHz, mały bufor 512 bitów) i generując falę dźwiękową bezpośrednio do pamięci RAM przy użyciu tablic matematycznych.

```
pygame.mixer.pre_init(44100, -16, 2, 512)

def generate_beep(self, frequency, duration):

      sample_rate = 44100

      n_samples = int(round(duration * sample_rate))

      buf = array.array('h')

      for i in range(n_samples):

            t = float(i) / sample_rate value = int(32767 * 0.5 *
            math.sin(2.0 * math.pi *        frequency * t))

            buf.append(value) # Lewy kanał

            buf.append(value) # Prawy kanał

      returnpygame.mixer.Sound(buffer=buf)
```

4.6 Interfejs graficzny i adaptacyjne okno

Zamiast sztywnych wielkości, do rysowania elementów użyto dynamicznych współrzędnych wyliczanych na podstawie aktualnego rozmiaru okna. Podanie flagi pygame.RESIZABLE pozwala użytkownikowi modyfikować rozmiar ramki lub przejść do trybu pełnoekranowego za pomocą systemowych ikon okna, a wykresy i teksty pozostają na środku.

```
offset_x = (self.width - self.base_width) // 2 offset_y =
(self.height - self.base_height) // 2
```

```
chart_x, chart_y = 150 + offset_x, 370 + offset_y
```

5. Logika programu

5.1 Algorytm działania programu

1. Uruchomienie aplikacji i inicjalizacja modułów graficznych oraz pre-inicjalizacja bufora audio,
2. Utworzenie głównego okna z zachowaniem proporcji i skalowalności,
3. Oczekiwanie w Menu na wybór trybu przez użytkownika,
4. Przejście do stanu INSTRUCTION przypisanego do wybranego modułu,
5. Uruchomienie maszyny stanów w pętli dla fazy TRAINING (3 iteracje z losowym przerwaniem i zrzucaniem wyników),
6. Uruchomienie maszyny stanów dla fazy TEST (5 iteracji z zapisem do zmiennej tablicowej self.results),
7. Przetworzenie tablicy wyników przy użyciu modułu statystycznego,
8. Przejście do stanu RESULTS - wyrenderowanie ocen, wygenerowanie wektorowego wykresu liniowego,
9. Oczekiwanie na powrót do punktu 3 (Menu główne).

5.2 Sposób działania logiki pomiarowej

Czas odświeżania głównej pętli (Polling Rate) został sztucznie podniesiony do 500 klatek na sekundę (self.fps = 500). Dzięki temu program skanuje bufor klawiatury systemowej z precyzją 2 milisekund. Jeśli wykryte zostanie naciśnięcie klawisza, czas jest odczytywany za pomocą time.perf_counter() - jest to funkcja korzystająca ze sprzętowego licznika cykli procesora (High-Resolution Performance Counter), omijająca niedokładności tradycyjnego zegara systemowego (tzw. "wall clock").

6. Dyskusja wyników

6.1 Zalety aplikacji

Do głównych zalet napisanego programu można zaliczyć:

- **Bardzo wysoka precyzja pomiarowa:** Zastosowanie perf_counter i 500 FPS gwarantuje rejestrowanie czasów reakcji niemal z laboratoryjną dokładnością (na poziomie błędu rzędu 2 ms).
- **Rozbudowana analityka:** Aplikacja wylicza nie tylko średnie czasy, ale bada stabilność reakcji (odchylenie standardowe) oraz skupienie na zadaniu (rozróżnia falstarty i pomyłki w zadaniach z wyborem).
- **Pełna skalowalność wizualna:** Algorytmy renderujące teksty i rysujące wykresy dostosowują się do wielkości okna (w tym pracy na pełnym ekranie na zewnętrznych monitorach).
- **Modularność:** Maszyna stanów umożliwia w przyszłości dodawanie nowych, bardziej złożonych testów (np. test Go/No-Go) dodając zaledwie jeden nowy warunek logiczny.

6.2 Wady aplikacji

Wśród ograniczeń napisanego oprogramowania można wymienić:

- **Systemowe opóźnienie sprzętowe:** Aplikacja, jako uruchomiona na domowym systemie operacyjnym ogólnego przeznaczenia (macOS/Windows), zawsze będzie obciążona ukrytym opóźnieniem związanym ze sprzętem (odświeżanie matrycy monitora, buforowanie klawiatury, protokół Bluetooth). Faktyczne czasy biologicznej reakcji mogą być o 30-100 ms krótsze niż wyniki wskazane przez program. Aby program działał obiektywnie, kandydaci muszą być badani na dokładnie tym samym zestawie sprzętowym.
- **Tymczasowość danych:** Po powrocie do Menu głównego tablica z wynikami jest czyszczona (self.results = []). Program nie prowadzi archiwizacji i nie zapisuje długoterminowej historii pacjentów.

6.3 Możliwe kierunki rozwoju

W przyszłości aplikację można by rozbudować o:

- Moduł eksportu danych, pozwalający wygenerować raport PDF lub zapisać wiersz do pliku CSV (Excel) z danymi, datą i czasem podsumowującym badanego kandydata,

- Funkcję Kalibracji (Offset) w opcjach programu, która pozwalałaby laborantowi ręcznie odjąć stały błąd (Hardware Lag) charakteryzujący dany zestaw komputerowy, by podawane czasy były bliskie rzeczywistej biologii człowieka.
- Możliwość samodzielnego zdefiniowania przez operatora liczby prób w teście z poziomu menu głównego (np. suwak od 5 do 30 prób).